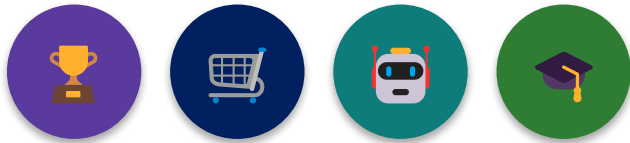


"The capstone is where every pattern in this course has to work together, at once, for real."

SUPPLEMENTARY · MODULE 13

Capstone Integration and Course Close

Bring every pattern together in one retail microservices platform: design, build, secure, observe, and operate it, then expose it safely to AI agents through MCP.



CAPSTONE BUSINESS SCENARIO · CAPSTONE FUNCTIONAL REQUIREMENTS · CAPSTONE NONFUNCTIONAL REQUIREMENTS

THE BUSINESS PROBLEM

- A growing retail company's monolithic system cannot scale, deploy independently, or adopt AI safely
- The capstone replaces it with independently deployable microservices on OpenShift

OPERATIONAL AND AI REQUIREMENTS

- Manage inventory and stock levels; coordinate fulfillment and shipping
- Expose operations to AI agents through MCP with human approval for sensitive actions

CORE CAPABILITIES NEEDED

- Product catalog, order management, inventory, fulfillment and shipping, returns and refunds
- Customer support, analytics and reporting, and AI agents that act on the customer's behalf

QUALITY ATTRIBUTES

- High availability and resilience through peak shopping traffic
- Low latency for catalog browsing and checkout; horizontal scalability under load

CUSTOMER-FACING REQUIREMENTS

- Browse and search the product catalog; place, track, and cancel orders
- Process payments and refunds; manage returns; contact support

TRUST AND COMPLIANCE ATTRIBUTES

- **Security by default:** OAuth2/OIDC, least privilege, encrypted data
- Full observability and auditability for every service and every agent action

KEY TAKEAWAY A retail company needs a modern platform for browsing products, placing orders, tracking inventory, fulfilling shipments, handling returns, and supporting customers -- with AI agents automating routine operations. • Every functional requirement traces back to a step in the customer's journey, from browsing the catalog through post-purchase support. • Nonfunctional requirements -- availability, latency, scalability, security, and observability -- decide the architecture just as much as the feature list does.

Capstone Service Boundaries

Decompose the retail platform into independently deployable services aligned to **business capabilities** and **data ownership**.

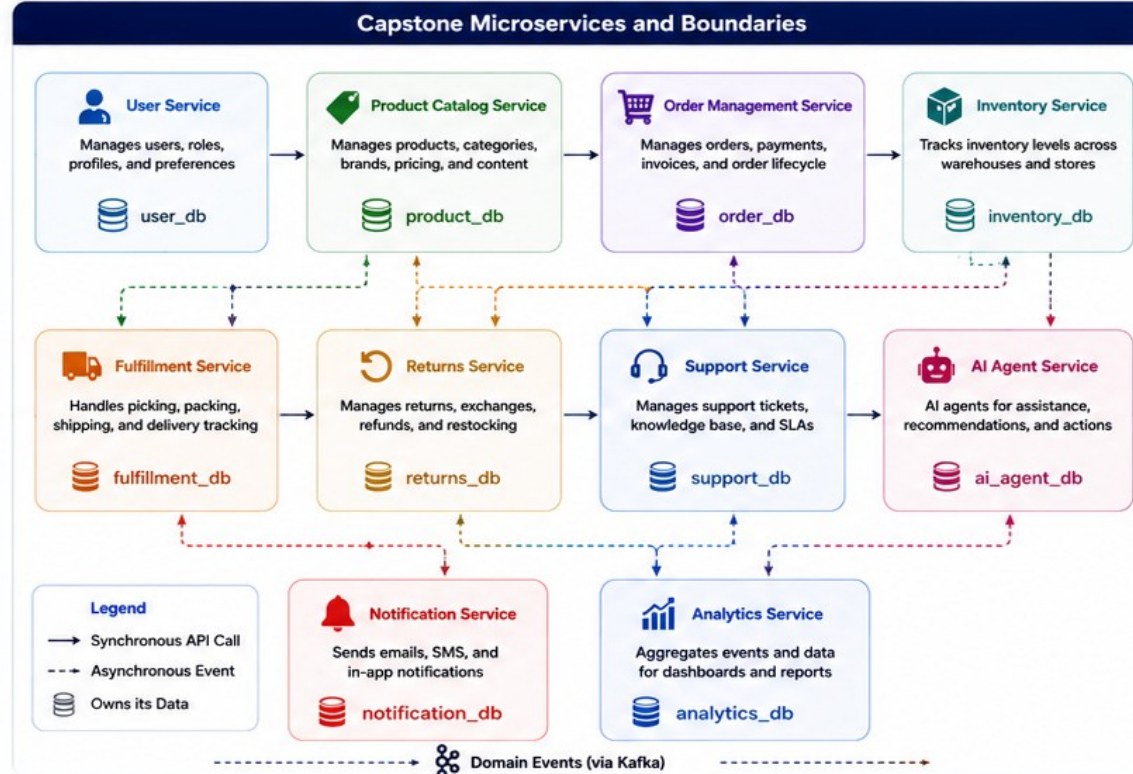


Design Principles

- ✓ Align services to business capabilities
- ✓ High cohesion within a service
- ✓ Low coupling between services
- ✓ Own your data (Database per Service)
- ✓ Independently deployable and scalable
- ✓ Communicate via APIs and events
- ✓ Evolutionary design – iterate and improve

Business Capability Mapping

Business Capability	Service(s)
Customer Management	User Service
Product Discovery & Catalog	Product Catalog Service
Ordering & Checkout	Order Management Service
Inventory Visibility	Inventory Service
Fulfillment & Shipping	Fulfillment Service
Returns & Refunds	Returns Service
Customer Support	Support Service
AI Assistance	AI Agent Service
Notifications	Notification Service
Analytics & Reporting	Analytics Service



Boundaries & Contracts

- ✓ Each service exposes a well-defined API
- ✓ No direct database access across services
- ✓ Data consistency achieved via events and eventual consistency
- ✓ Clear ownership of data and business logic
- ✓ Versioned APIs for backward compatibility
- ✓ Anti-corruption layers for external systems

Cross-Cutting Services

- API Gateway**: Single entry point for all clients
- Service Discovery**: Dynamic service registration and lookup
- Config Service**: Centralized configuration management
- Security & IAM**: Authentication, authorization, and policies
- Observability**: Logs, metrics, traces across all services

Data Ownership Summary

Service	Owns Data	Service	Owns Data
User Service	user_db	Returns Service	returns_db
Product Catalog Service	product_db	Support Service	support_db
Order Management Service	order_db	AI Agent Service	ai_agent_db
Inventory Service	inventory_db	Notification Service	notification_db
Fulfillment Service	fulfillment_db	Analytics Service	analytics_db

Key Benefits

- ✓ Independent deployment and scaling
- ✓ Technology and data diversity
- ✓ Fault isolation and improved resilience
- ✓ Team autonomy and clear ownership
- ✓ Faster delivery and easier evolution



Considerations

- ⚠️ Avoid over-segmentation (too many tiny services)
- ⚠️ Define boundaries that minimize cross-service dependencies
- ⚠️ Plan for data synchronization and consistency
- ⚠️ Review and refactor boundaries as the system evolves

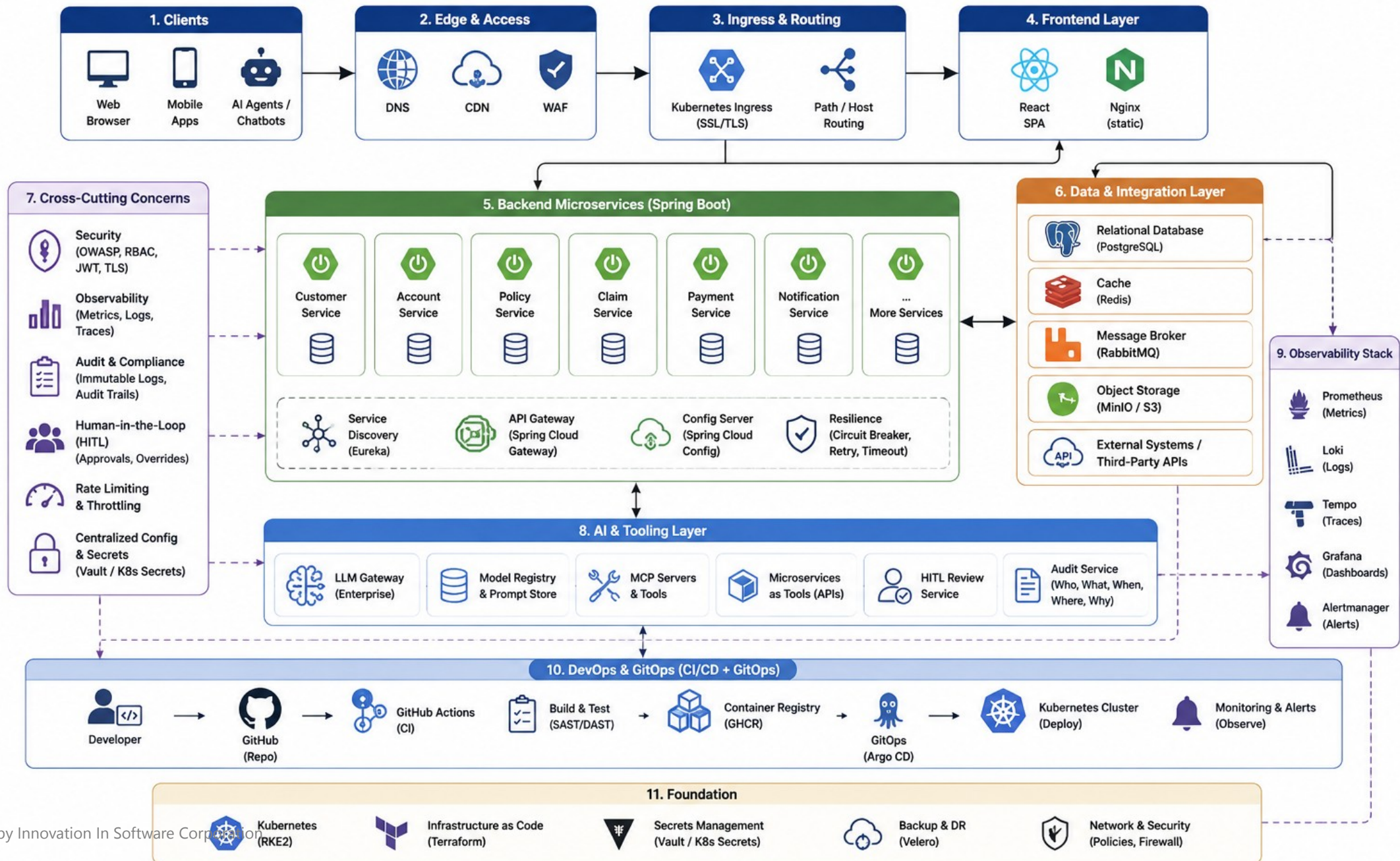


Key Takeaway: Well-defined service boundaries aligned to business capabilities and data ownership are the foundation

of a scalable, maintainable, and resilient microservices architecture.



Capstone Architecture Recap



CAPSTONE DOMAIN AND DATA MODEL · CAPSTONE REST API CONTRACTS · CAPSTONE MESSAGING ARCHITECTURE



CORE ENTITIES

- Customer owns Orders; Order owns OrderItems referencing Products
- Category groups Products; Payment and Shipping each reference an Order



OWNERSHIP DISCIPLINE

- Each service owns its slice of this model -- no cross-service foreign keys
- Cross-service references travel as IDs in events and API payloads, not joins



CONTRACT CONVENTIONS

- Versioned base paths, plural resource nouns, standard HTTP verbs and status codes
- Consistent error response shape across every service



EXAMPLE CONTRACTS

- **Order Service:** POST /orders, GET /orders/{id}, GET /orders/{id}/status
- **Payment Service:** POST /payments;
- **Inventory Service:** GET /inventory/{sku}



EVENT-DRIVEN FLOW

- Order Service publishes OrderCreated; Payment and Inventory consume it independently
- Shipping Service reacts to PaymentConfirmed to begin fulfillment



WHY EVENTS, NOT CALLS

- Producers and consumers scale and fail independently
- New consumers (like Analytics or AI Agent services) subscribe without changing producers

KEY TAKEAWAY The core transactional slice of the platform -- Customer, Catalog/Category/Product, Order/OrderItem, Payment, and Shipping -- is where most of the capstone's implementation effort concentrates.

- A consistent REST contract style across all six core services -- resource paths, verbs, status codes, and error bodies -- keeps the platform predictable for both human and AI consumers.
- Kafka topics for order, payment, inventory, and shipping events let services react asynchronously instead of chaining synchronous calls across the order lifecycle.

CAPSTONE DATABASE OWNERSHIP · CAPSTONE TRANSACTION STRATEGY · CAPSTONE CACHING STRATEGY



OWNERSHIP MAP

- Order Service owns order_db; Payment Service owns payment_db; Inventory owns inventory_db
- Customer and Shipping services each own their own database in the same pattern



WHY A SAGA

- No distributed two-phase commit across independently owned databases
- Each local transaction is followed by an event that triggers the next step



THE RULE THAT PROTECTS IT

- No shared schemas, no cross-service joins, no direct database access from another service



WHAT GETS CACHED

- Product catalog and pricing data with longer TTLs
- Inventory availability with short TTLs, invalidated on every stock change



THE ORDER SAGA

- Order created (pending) -> payment authorized -> inventory reserved -> order confirmed
- **A failure at any step triggers compensating actions:** release inventory, refund payment, cancel order



CACHE DISCIPLINE

- **Cache-aside pattern:** read cache, fall back to database, populate cache on miss
- Never cache payment or order state -- correctness there outweighs latency

KEY TAKEAWAY Order, Payment, Customer, Inventory, and Shipping each own a private database; any other service that needs their data asks through an API or listens for an event. • Placing an order touches Order, Payment, and Inventory services -- a choreographed saga with compensating actions keeps them consistent without a distributed transaction. • Lecture only (not in Labs 1-5): enterprises often cache read-heavy data in Redis. This capstone keeps PostgreSQL as the system of record for Account, Transaction, and Risk.

CAPSTONE RESILIENCE STRATEGY · CAPSTONE SECURITY MODEL · CAPSTONE OBSERVABILITY STRATEGY



RESILIENCE4J POLICIES IN USE

- Timeouts and bounded retries with backoff on every outbound call
- Circuit breakers open on sustained failure; bulkheads isolate thread pools per dependency



SECRETS AND ACCESS

- OpenShift Secrets for credentials, keys, and tokens -- never hardcoded
- Role-based access control scoped to least privilege for every service account



GRACEFUL DEGRADATION

- Fallbacks return cached or partial data instead of failing the whole request
- Rate limiting protects services from traffic spikes



THREE PILLARS

- **Logs:** structured, centralized, correlation IDs on every entry
- **Metrics:** RED metrics per service, business and technical, alerted on symptoms
- **Traces:** distributed tracing across every service boundary



LAYERS OF DEFENSE

- OAuth2/OIDC and JWT for user and agent authentication and authorization
- mTLS between services; network policies restrict which services can call which



OUTCOMES

- Faster mean time to resolution and earlier detection of customer-facing issues

KEY TAKEAWAY Every call between Order, Payment, Inventory, Shipping, and Notification services carries a timeout, a bounded retry, and a circuit breaker so one failing dependency degrades gracefully instead of cascading. • User-facing traffic authenticates through OAuth2/OIDC and JWTs; service-to-service traffic is mutually authenticated; every credential and secret lives in OpenShift Secrets, never in code. • Structured logs, Micrometer/Prometheus metrics, and OpenTelemetry traces, correlated by a shared trace ID, let the team find the root cause of any capstone incident quickly.

CAPSTONE CONTAINERIZATION STRATEGY · CAPSTONE OPENSIFT DEPLOYMENT MODEL · CAPSTONE CI-CD PIPELINE



BUILD PRINCIPLES

- **Multi-stage builds:** compile in one stage, run from a minimal JRE-Alpine image
- Immutable images, one service per container, non-root execution



OPENSIFT READINESS

- Health checks, externalized config via ConfigMaps and Secrets, small attack surface



DEPLOYMENT SHAPE

- Edge users -> OpenShift Route (TLS, WAF, rate limiting) -> API Gateway -> services
- **Data layer:** PostgreSQL per service. Integration: Kafka (AMQ Streams stand-in). MongoDB/Redis/RabbitMQ are lecture comparisons



PLATFORM SERVICES USED

- Service mesh, Prometheus, Elastic/Loki, Jaeger/Tempo, Alertmanager, and a secured registry
- Rolling, blue/green, and canary deployment patterns; HPA-based autoscaling



THE NINE STAGES

- Source -> Build -> Quality Gates -> Test -> Package -> Scan -> Push -> Deploy -> Verify & Monitor
- Quality, test, security, and deployment gates block promotion on failure



ENVIRONMENT PROMOTION

- Dev auto-deploys on merge; QA/Staging mirrors production for pre-release validation
- Production releases are controlled, monitored, and alerted end-to-end

KEY TAKEAWAY Every capstone service builds through the same multi-stage Dockerfile pattern -- minimal JDK base image, non-root user, externalized configuration -- so images stay small, secure, and consistent. • External traffic enters through an OpenShift Route with TLS termination and WAF/rate limiting, then a Spring Cloud Gateway routes it to User, Order, Inventory, and Payment services inside a dedicated project namespace. • Source, build, quality gates, test, package, scan, push, deploy, and verify-and-monitor form one pipeline that promotes a change through Dev, QA/Staging, and Production only after every gate passes.

CAPSTONE GIT WORKFLOW · CAPSTONE AUTOMATED TESTING STRATEGY



BRANCH ROLES

- **main:** production-ready, tagged releases develop: integration, always deployable
- **feature/*:** new work off develop release/*: stabilization hotfix/*: urgent fixes off main



THE PYRAMID

- Unit (40%) and component/service tests (30%) run fast and in isolation
- Integration and contract tests (20%) verify messaging and service interactions
- End-to-end tests (10%) validate critical user journeys across the platform



RULES THAT PROTECT THE HISTORY

- Pull requests and at least one approval required for every merge to develop and main
- All checks must pass; conventional commit messages; no direct pushes to main



QUALITY GATES

- Coverage thresholds, zero critical/high vulnerabilities, 100% contract-test success required to merge

KEY TAKEAWAY A protected main branch, an always-deployable develop branch, short-lived feature branches, and release/hotfix branches keep the capstone's history clean and every merge reviewed. • Most capstone tests are fast unit and component tests; a smaller layer of integration and contract tests verifies service interactions; a thin end-to-end layer validates full user journeys.

Required Demonstration Scenarios

1 User Registration & Profile Management



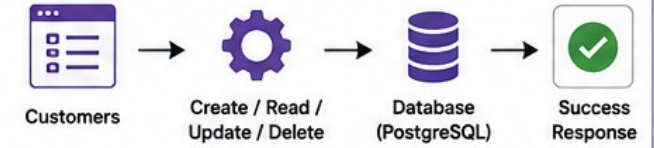
- Register a new user
- Update profile information
- Validate input and handle errors

2 Secure Authentication & Authorization



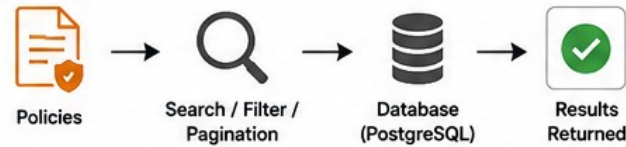
- Login with valid and invalid credentials
- Access protected endpoints by role
- Demonstrate token expiration & refresh

3 Customer Management (CRUD)



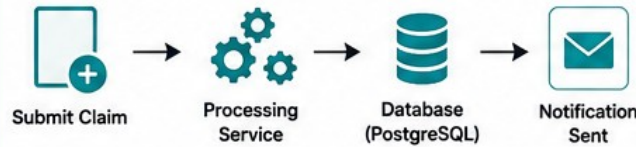
- Create a customer
- Read and update customer details
- Delete a customer (soft delete)

4 Policy Management & Search



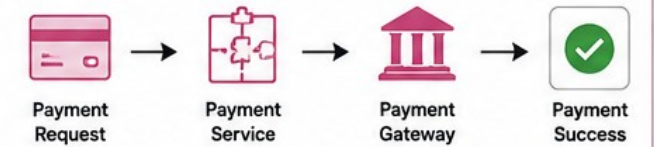
- Create and manage policies
- Search with filters and pagination
- Validate business rules and constraints

5 Claim Submission & Processing



- Submit a claim with documents
- Process claim and update status
- Trigger notifications on status changes

6 Payment Processing Integration



- Initiate a payment for an approved claim
- Handle success and failure responses
- Verify transaction records

7 Role-Based Access Control (RBAC)



- Test multiple roles (Admin, Staff, Customer)
- Verify allowed and denied access
- Ensure least privilege enforcement

8 Human-in-the-Loop (HITL) Workflow



- Reviewer receives items for review
- Approve, reject, or request more information
- System updates status and audit trail

9 Audit Logging & Observability



- Capture key events and changes
- View logs in dashboards
- Trigger alerts on critical events



Success Criteria:



Functional
Correctness



Security &
Authorization



Performance &
Reliability



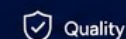
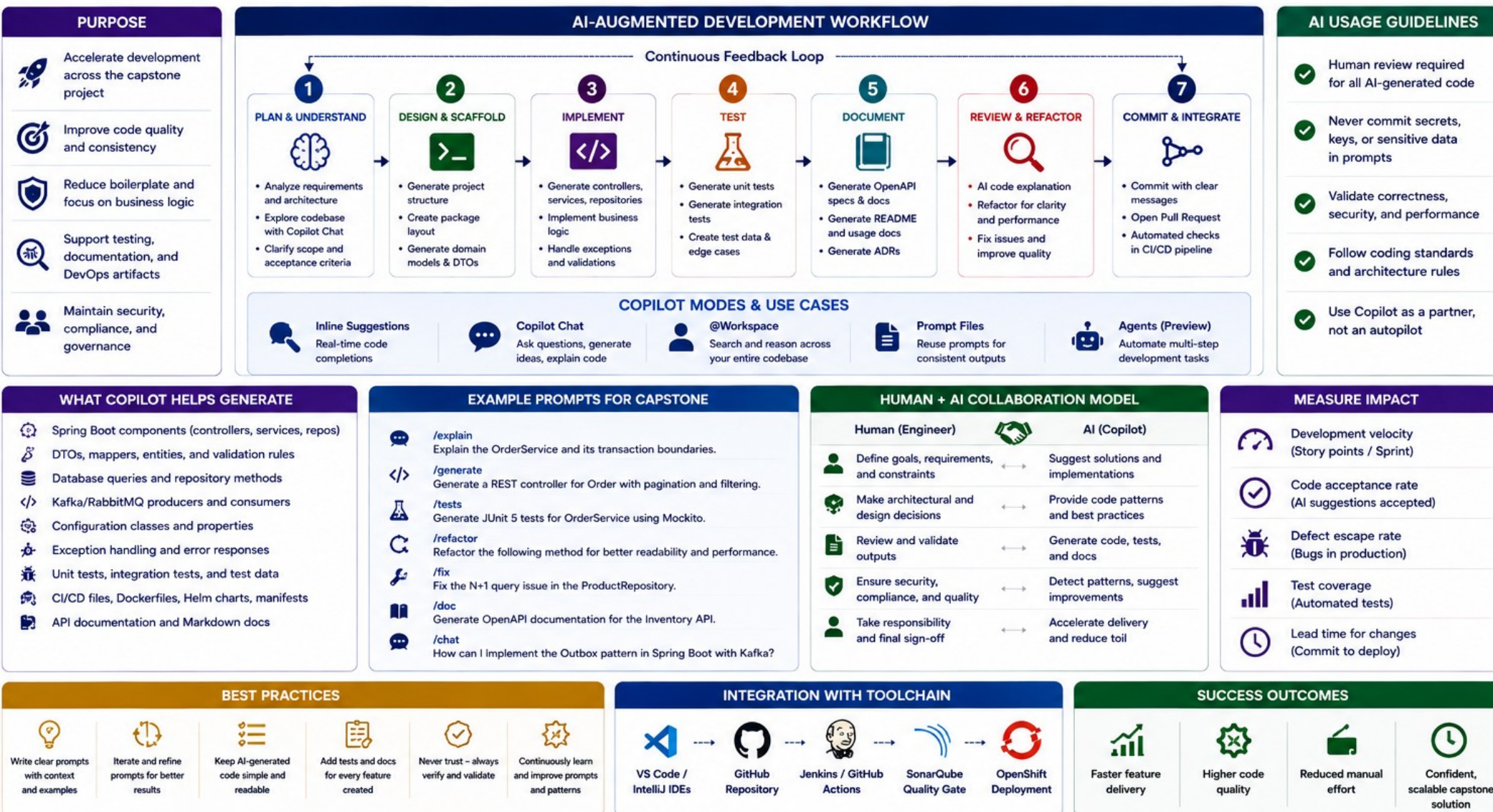
Data Integrity &
Consistency



Observability &
Auditability

Capstone GitHub Copilot Workflow

AI-powered development with human expertise — build faster, smarter, and with confidence.



CAPSTONE AGENTIC AI WORKFLOW · CAPSTONE MCP SERVER DESIGN · CAPSTONE HUMAN-APPROVAL WORKFLOW



THE SIX-STEP LOOP

- User request -> agent orchestrator (intent, planning) -> tool selection -> execute actions
- Aggregate and reason over results -> generate the final response to the user



GUARDRAILS AND SAFETY

- Least-privilege access to tools; human approval for destructive actions
- Prompt-injection protection, rate limiting, and comprehensive audit logging



MCP SERVER LAYERS

- **MCP interface layer:** tool discovery, schema management, request routing
- Capabilities registry (tools, resources, prompts) and an adapter layer to backend service APIs



POLICY AND SECURITY ENFORCEMENT

- Authentication and authorization, permission checks, rate limiting, and audit logging at the boundary
- Strongly typed, versioned tool schemas reduce ambiguity for agents



THE FIVE-STEP FLOW

- Agent plans action -> risk evaluation and policy check -> human review (if required) -> execute -> audit, notify, monitor
- Low-risk actions auto-approve and execute; high-risk actions route to a human



BEST PRACTICES

- Define clear triggers by risk and impact; show full context and rationale to the reviewer
- Set review time limits and log every decision for traceability

KEY TAKEAWAY A six-step agent loop -- user request, orchestrator, tool selection, execute actions, aggregate and reason, response -- lets an AI agent complete real capstone workflows, like cancelling an order and issuing a refund, safely and observably. • Tools, resources, and prompts behind an MCP interface layer -- with authentication, permission checks, rate limiting, and audit logging enforced at the boundary -- let agents discover and invoke capstone operations safely. • A risk-evaluation step decides whether an agent's proposed action executes automatically or waits for a human reviewer to approve, request changes, or reject it before anything touches production data.

CAPSTONE AUDIT AND COMPLIANCE REQUIREMENTS · CAPSTONE FAILURE SCENARIOS · CAPSTONE PERFORMANCE REQUIREMENTS



WHAT EVERY EVENT CAPTURES

- Actor (user or agent), action, resource, timestamp, correlation ID, and result
- Enriched with identity, service, request parameters, and success/failure



FAILURE CATEGORIES TO TEST

- **Service failures:** crash, unresponsive pod Data layer failures: database down, connection pool exhaustion
- **Network issues:** latency, packet loss Messaging failures: broker down, consumer backlog
- **Resource exhaustion:** high CPU/memory, disk full, thread pool saturation



COMPLIANCE CONTROLS

- Write-once, append-only storage with encryption at rest and in transit
- Defined retention periods and least-privilege access to audit data



EXAMPLE TARGETS

METRIC	TARGET
API latency (p95)	≤ 300 ms
Throughput	≥ 1,000 requests/sec
Error rate	≤ 0.1%
Availability	≥ 99.9%

KEY TAKEAWAY Every audit event records who did what, when, where, why, and the outcome, written to an append-only store with encryption and retention controls so the platform can withstand a compliance review. • The capstone's resilience design is only proven once it's been tested against real failure scenarios: a crashed pod, an unreachable database, packet loss, a down message broker, or exhausted CPU and memory. • Explicit performance targets -- response time percentiles, throughput, error rate, and availability -- give the capstone team a measurable bar instead of a vague sense of "fast enough."

CAPSTONE PRODUCTION-READINESS CRITERIA · CREATING THE SPRING BOOT SERVICES



READINESS CHECKLIST

- All automated tests and security scans pass; performance targets are met
- Observability, alerting, and rollback are proven; audit and compliance controls verified
- Runbooks and on-call ownership are documented for every service



PROJECT SKELETON

- spring-boot-starter-web, -data-jpa, -validation, and -actuator as the baseline dependencies
- **Layered packages:** controller, service, repository, dto, entity, config



CONFIGURATION DISCIPLINE

- application.yml per profile (dev, qa, prod); no environment values hardcoded
- Each service (like OrderService) is generated the same way for consistency

KEY TAKEAWAY Production readiness for the capstone means every quality gate has passed, every failure scenario has been tested, observability is in place, and the team has a runbook -- not just that the demo works. • Order, Payment, Customer, Inventory, Shipping, and Notification services all start from the same Spring Boot skeleton: Web, Data JPA, Validation, and Actuator starters, layered packages, and profile-based configuration.

IMPLEMENTING REST API CONTRACTS · IMPLEMENTING PERSISTENCE AND MIGRATIONS



IMPLEMENTATION PATTERN

- @RestController delegates to a service layer; @Valid enforces request DTO validation
- 201 Created with Location header on POST; 404 with a structured error body on not-found



MIGRATION DISCIPLINE

- Versioned, checked-in SQL migration scripts per service, applied on startup
- spring.jpa.hibernate.ddl-auto=validate in every environment beyond local development



REPOSITORY LAYER

- Spring Data JPA repositories per aggregate root, with indexes matching real query patterns

KEY TAKEAWAY Implementing a contract means a thin @RestController, @Valid request DTOs, a consistent status-code convention, and a shared error-response shape -- the same pattern on every capstone service. • Every capstone service manages its own schema through versioned Flyway or Liquibase migrations, never through Hibernate's auto-DDL, so schema changes are reviewable and repeatable across environments.

IMPLEMENTING SERVICE-TO-SERVICE COMMUNICATION · IMPLEMENTING EVENT-DRIVEN INTEGRATION



THE CALL PATTERN

- WebClient or a Feign-style declarative client per downstream service
- Every call wrapped in @CircuitBreaker, @Retry, and a bounded @TimeLimiter



PRODUCER AND CONSUMER PATTERN

- KafkaTemplate publishes domain events after the local transaction commits
- @KafkaListener consumers are idempotent, tracking processed event IDs



FAILURE HANDLING

- Dead-letter topics and retry policies for poison messages

KEY TAKEAWAY Synchronous calls between capstone services -- Order calling Inventory to check stock, for example -- go through a declarative client wrapped in a timeout, retry, and circuit breaker, never a bare HTTP call. • Order Service publishes OrderCreated with Spring Kafka; Payment and Inventory services consume it with idempotent, at-least-once handlers that tolerate redelivery without double-processing.

IMPLEMENTING DISTRIBUTED DATA CONSISTENCY · ADDING REDIS CACHING · ADDING RESILIENCE4J POLICIES



THE OUTBOX PATTERN

- Order and its OrderCreated event row are written in one local transaction
- A separate relay process publishes outbox rows to Kafka and marks them sent



IMPLEMENTATION PATTERN

- @Cacheable on catalog read methods; @CacheEvict on writes that change cached data
- **Lecture only:** TTL per cache region -- not configured in the capstone labs



COMPENSATION LOGIC

- Saga compensating handlers implement the rollback steps designed in Topic 659



POLICIES IN CODE

- @CircuitBreaker(name="inventoryService", fallbackMethod="...") around risky calls
- @Retry with exponential backoff; @Bulkhead to isolate thread pools per dependency

KEY TAKEAWAY Writing the order and its outgoing event in the same local transaction -- the outbox pattern -- prevents the classic dual-write bug where a database commit succeeds but the event never gets published, or the reverse. • Lecture only: @Cacheable / @CacheEvict over Redis is a common enterprise pattern. Labs 1-5 do not add a cache. • @CircuitBreaker, @Retry, @TimeLimiter, and @Bulkhead annotations from Resilience4j, configured per downstream dependency, are how the resilience strategy from Topic 661 actually gets enforced at runtime.

ADDING OAUTH2 AND OIDC SECURITY · ADDING LOGS, METRICS, AND TRACES · GENERATING TESTS WITH GITHUB COPILOT



IMPLEMENTATION PATTERN

- spring-boot-starter-oauth2-resource-server validates JWT signature, issuer, and audience
- @PreAuthorize enforces scopes and roles at the method level



EXAMPLE PROMPT

- "Generate JUnit 5 tests for OrderService using Mockito, including edge cases"



INSTRUMENTATION IN CODE

- Structured logging with trace ID, span ID, and service name in every entry
- Micrometer counters and timers exposed via /actuator/prometheus
- OpenTelemetry SDK auto-instruments HTTP, JPA, and Kafka calls



STILL THE DEVELOPER'S JOB

- Verify each assertion is meaningful, not just present
- Add missing edge cases Copilot didn't think to cover

KEY TAKEAWAY Each capstone service is configured as an OAuth2 resource server that validates incoming JWTs, checks scopes and roles, and rejects unauthenticated or under-privileged requests before they reach a controller. • Structured JSON logs with correlation IDs, Micrometer metrics exposed through Actuator, and an OpenTelemetry SDK propagating trace context are how every capstone service actually becomes observable. • A well-scoped prompt asking Copilot to generate JUnit 5 tests for OrderService, including edge cases, gets a fast first draft -- the developer's job is to check every assertion actually verifies the right behavior.

REVIEWING AND CORRECTING AI-GENERATED CODE · PACKAGING SERVICES AS CONTAINERS · BUILDING THE CI-CD PIPELINE



COMMON CORRECTIONS NEEDED

- Deeper validation than a simple null check; complete field mapping
- Explicit transactional boundaries; proper exception handling



THE BUILD IN PRACTICE

- Build stage compiles with Maven; runtime stage copies only the built JAR
- USER spring:spring; EXPOSE 8080; ENTRYPOINT tuned for container memory limits



PIPELINE AS CODE

- **Awareness:** a Jenkinsfile would mirror those stages. Lab 4 uses run-pipeline-locally.ps1
- SonarQube quality gate, Trivy image scan, and OpenShift deployment all wired into the pipeline

KEY TAKEAWAY A null check that's too shallow, a mapper that misses a field, a missing transactional boundary -- Copilot's first draft on a capstone service still needs a human reviewer checking correctness, security, and maintainability. • Applying the containerization strategy from Topic 664 to real code: a multi-stage Dockerfile per service, an OpenJDK-Alpine runtime image, a non-root user, and health checks wired to Spring Boot Actuator. • Awareness: enterprises often encode that pipeline in a Jenkinsfile. Lab 4 participants run a prepared script (scan / SBOM / sign) -- they do not install Jenkins.

DEPLOYING THE CAPSTONE TO OPENSIFT · SCALING CAPSTONE SERVICES · IMPLEMENTING THE AGENTIC AI WORKFLOW



THE DEPLOYMENT SEQUENCE

- Push image -> deploy to OpenShift -> run database migrations -> health and readiness checks
- Traffic only routes to pods that pass their readiness probe



SCALING MECHANICS

- HPA targets per-service CPU/memory utilization or a custom metric (like Kafka lag)
- Stateless services scale horizontally without session affinity concerns



EXAMPLE: "CANCEL ORDER #1245 AND REFUND"

- Agent understands intent, retrieves order details, calls Order Service to cancel
- Agent publishes a RefundRequested event; Payment Service processes the refund
- Agent confirms the refund and notifies the user; every step is logged and audited

KEY TAKEAWAY Deploying to OpenShift means a Deployment with liveness/readiness probes per service, a Route with TLS for external access, ConfigMaps and Secrets for configuration, and database migrations run before traffic is allowed in. • A Horizontal Pod Autoscaler watching CPU, memory, or a custom queue-depth metric adds or removes replicas of Order, Inventory, and other capstone services automatically as real demand rises and falls. • Implementing the agent workflow from Topic 670 means building the orchestrator that plans and reasons, wiring it to a tool registry (MCP), and enforcing RBAC, rate limits, and human-in-the-loop policies at every step.

EXPOSING CAPSTONE OPERATIONS THROUGH MCP · ADDING HUMAN-IN-THE-LOOP CONTROLS



TOOL DEFINITION IN PRACTICE

- Each tool declares name, description, input schema, output schema, and required fields
- MCP server validates input, authorizes the caller, invokes the backend service, returns a structured result



IMPLEMENTATION PRACTICES

- Define clear triggers by risk and impact (e.g., refunds over a threshold)
- Provide full context, rationale, and a simple UI for the reviewer's decision
- Set SLAs for review response and escalate automatically if they're missed

KEY TAKEAWAY Exposing capstone operations through MCP means wrapping each one -- create_order, get_order, update_inventory, process_payment, send_notification -- in a strongly typed tool with input/output schemas, least-privilege access, and audit logging. • Adding human-in-the-loop controls in practice means defining which actions auto-approve, giving reviewers full context and a simple approve/request-changes/reject decision, and logging every outcome for audit.

RECORDING AGENT AND USER AUDIT EVENTS · RUNNING END-TO-END TESTS · RUNNING FAILURE AND RECOVERY TESTS



THE AUDIT EVENT SHAPE

- eventId, timestamp, actor (user or agent), action, resource, service, correlationId, result
- Stored write-once in an append-only, encrypted, access-controlled audit store



EXAMPLE E2E SCENARIO

- Place an order from the web portal -> Order and Inventory services validate the transaction
- Payment service processes it; Notification service sends confirmation
- Test verifies order status, inventory update, and notification -- the full business outcome



EXAMPLE FAILURE TESTS

- Kill an Order Service pod and verify automatic restart and traffic recovery
- Simulate a database outage and validate graceful fallback, not data corruption
- Simulate broker downtime and validate dead-letter handling and recovery

KEY TAKEAWAY Every user and agent action writes a structured audit event -- actor, action, resource, timestamp, correlation ID, and outcome -- to an immutable, encrypted, access-controlled store.

- An end-to-end test for the capstone places an order through the UI, then verifies the order and inventory services, the payment service, and the notification service all reached the correct final state.
- Failure and recovery testing on the capstone means deliberately killing a service pod, simulating a database outage, or injecting high latency, then confirming automatic recovery and no data loss.

RUNNING SECURITY VALIDATION · RUNNING PERFORMANCE VALIDATION · DEMONSTRATING BLUE-GREEN OR CANARY DEPLOYMENT



EXAMPLE SECURITY CHECKS

- Scan container images for critical vulnerabilities; verify JWT signature, expiration, audience
- Test API endpoints for authorization bypass; check for SQL/NoSQL injection and XSS
- Ensure secrets are stored in a secure vault, never in code or config files



BLUE/GREEN

- Deploy to Green, run health and smoke tests, switch traffic, keep Blue as instant fallback



TYPES OF PERFORMANCE TESTS

- Load testing (expected traffic) and stress testing (find the breaking point)
- Spike testing (sudden traffic surges) and soak testing (sustained load over time)



CANARY

- Route 10% of traffic to the new version, monitor metrics, then ramp toward 100%

KEY TAKEAWAY Security validation for the capstone runs SAST, DAST, dependency scanning, container scanning, and secrets scanning, then checks the results against OWASP Top 10 risks before anything ships. • Performance validation runs load, stress, spike, soak, and scalability tests against the capstone, then checks the measured p95 latency, throughput, and error rate against the targets set in Topic 675. • A blue/green demo validates a new version in an idle Green environment before switching all traffic to it; a canary demo routes a small percentage of real traffic to the new version and watches metrics before ramping further.

DEMONSTRATING AUTOMATED ROLLBACK · DEMONSTRATING AGENT-TO-MICROSERVICE INTERACTION · CAPSTONE ARCHITECTURE REVIEW

EXAMPLE OUTCOME

- **Detected error rate of 6.5% (threshold: 2%)** -- automatic rollback executed
- Traffic returned to the previous stable version in under a minute; error rate recovered

SIX-LAYER REVIEW

- Clients & consumers -> ingress & edge (Route, API Gateway, security) -> six core microservices
- Data & messaging layer -> observability & operations -> cross-cutting capabilities (CI/CD, RBAC, DR)

THE DEMO FLOW

- User asks a question -> agent plans and identifies tools -> agent queries MCP for schemas
- Agent invokes tools; MCP routes to Order and Shipping services -> results are aggregated and returned

KEY TAKEAWAYS

- Event-driven, resilient by design, secure by default, observable at scale, AI-ready through MCP

KEY TAKEAWAY A rollback demo deploys a new version, deliberately breaches an error-rate or latency threshold, and shows the pipeline detect it, automatically route traffic back to the previous stable version, and verify recovery. • A live demo traces one user request through the agent, the MCP server's tool discovery and execution, the Order and Shipping services, and back -- showing the whole agentic path is secure, observable, and correct. • Stepping back across the whole platform -- clients, ingress and edge, the Order/Payment/Customer/Inventory/Shipping/Notification microservices layer, data and messaging, and observability -- confirms the architecture is business-aligned, resilient, secure, and AI-ready.

CAPSTONE CODE REVIEW · CAPSTONE DEMONSTRATION AND FEEDBACK



REVIEW FOCUS AREAS

- Correctness and edge cases; security (auth, input validation, secrets)
- Performance (queries, caching); readability, maintainability, and test coverage



DEMONSTRATION FLOW

- Introduction -> live demo -> key scenarios -> AI agent interaction -> observability -> summary



CHECKLIST HIGHLIGHTS

- No hardcoded values; AI-generated code reviewed just as carefully as human-written code

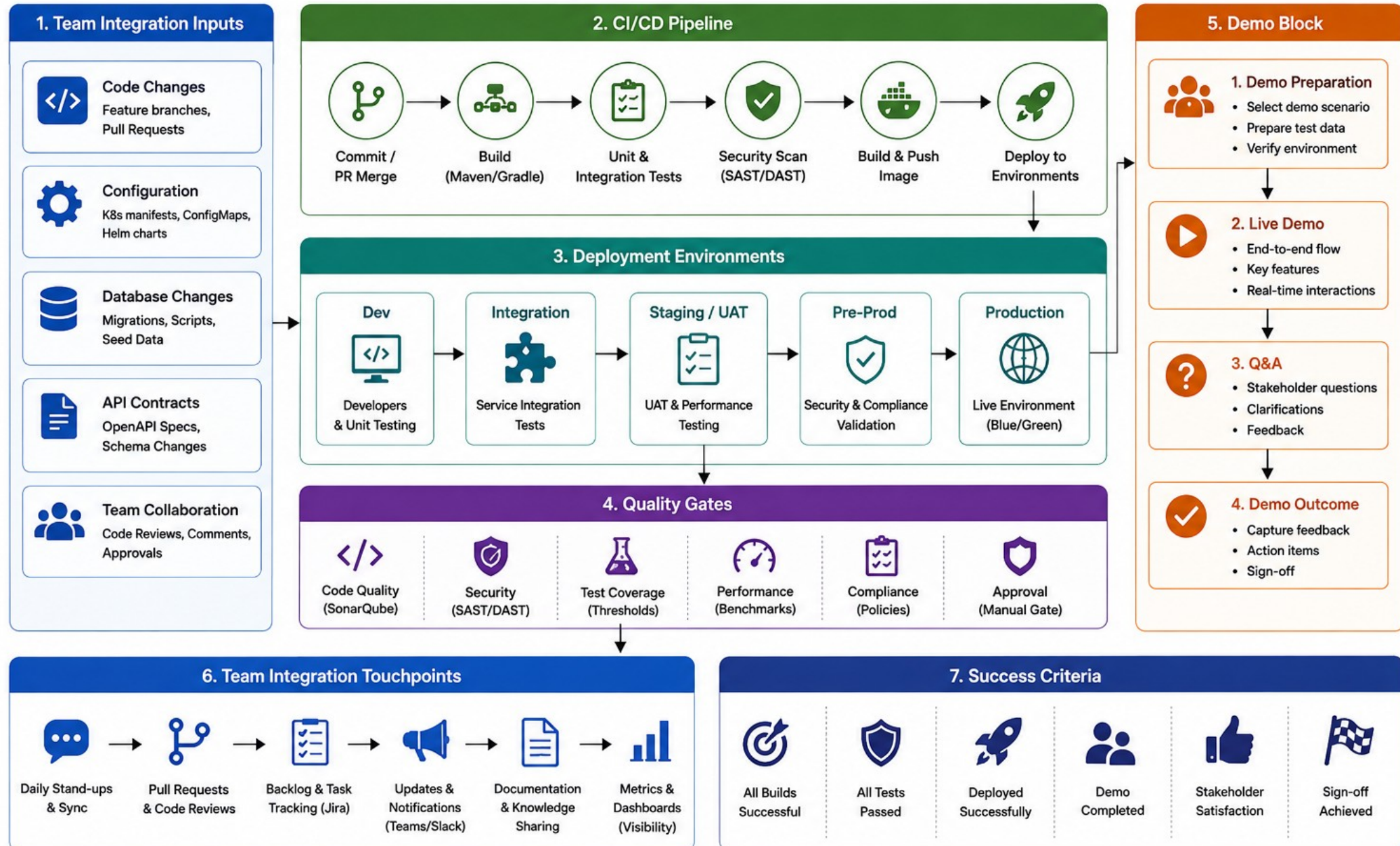


FEEDBACK PROCESS

- Collect feedback -> analyze and prioritize -> create action items -> track and improve

KEY TAKEAWAY A capstone code review walks a real Spring Boot method -- like `OrderService.createOrder` -- checking validation, mapping correctness, transactional boundaries, and response structure before approving the merge. • The final capstone demonstration walks the full solution end-to-end -- introduction, live demo, key scenarios, AI agent interaction, observability, and summary -- then captures structured feedback across functionality, reliability, security, usability, and overall value.

Team Integration and Demo Block



MICROSERVICES FUNDAMENTALS RECAP · SERVICE DESIGN RECAP · SPRING BOOT DEVELOPMENT RECAP



WHAT WE ESTABLISHED

- Why organizations move from monoliths to microservices, and the tradeoffs involved
- **Core characteristics:** independent deployability, decentralized data, business-capability alignment



WHAT WE ESTABLISHED

- Domain-driven boundaries, bounded contexts, and API contract design
- Versioning, idempotency, and status-code discipline as the law between services



WHAT WE ESTABLISHED

- **Production-shaped services from the start:** starters, profiles, Actuator, layered packages
- Externalized configuration as a requirement, not an afterthought

KEY TAKEAWAY Microservices trade a monolith's simplicity for independent deployability, technology flexibility, and fault isolation -- at the cost of distributed-systems complexity the rest of this course exists to manage. • Well-drawn service boundaries and disciplined REST API contracts are the design decisions that determine whether a microservices system stays maintainable or turns into a distributed monolith. • The Spring Boot habits built early in this course -- starters, profile-based configuration, Actuator, and externalized config -- are exactly the habits that let a service run unmodified on OpenShift later.

INTEGRATION AND MESSAGING RECAP

Course Recap
Day 5 • Module 29

Connecting services reliably, asynchronously, and at scale while maintaining loose coupling and resilience.



Effective integration patterns and messaging enable microservices to communicate reliably, handle failures gracefully, and stay independent.

1. COMMUNICATION PATTERNS



Synchronous (HTTP)

- REST APIs for request/response
- Simple to implement and understand
- Best for real-time interactions
- Use timeouts, retries, and circuit breakers



Asynchronous (Messaging)

- Queue or topic-based decoupled communication
- Improves scalability and fault tolerance.
- Best for events, notifications, and long-running tasks
- Use idempotency and proper delivery guarantees



Hybrid Integration

- Combine sync and async as needed
- Use API Gateway for external entry
- Choose the right pattern per use case

2. MESSAGING FUNDAMENTALS



Queues (P2P)

Point-to-point messaging between producer and single consumer.



Topics (Pub/Sub)

Publish once, subscribe many. Ideal for events and broadcasts.



Delivery Guarantees

At-most-once, at-least-once, exactly-once (under constraints).



Idempotent Consumers

Ensure safe processing of duplicate messages.



Retries & Redelivery

Handle transient failures with backoff and redelivery policies.



DLQ

Dead-letter queues isolate poison messages for later inspection.



Ordering & Partitioning

Maintain order where needed using partitions and keys.



Security

Secure brokers, topics, and messages with encryption and ACLs.

3. POPULAR MESSAGE BROKERS



- High throughput and scalability
- Topic-based pub/sub
- Partitioning for scale and ordering
- Strong replay and retention
- Great for streaming and analytics

VS



- Flexible routing with exchanges
- Queue-based (P2P) and pub/sub
- Rich routing and binding options
- Simpler for traditional messaging
- Great for task queues and RPC

4. RELIABILITY PATTERNS



Transactional Outbox

Guarantee events are published reliably.



Change Data Capture

Capture changes from DB and publish events.



Saga Pattern

Manage distributed transactions with compensating actions.



Idempotency Key

Prevent duplicate effects from reprocessed messages.

5. OBSERVABILITY FOR INTEGRATION



Metrics

Throughput, lag, errors, retries, and consumer performance.



Logs

Correlation IDs across services and messages.



Traces

End-to-end visibility for request and event flows.



Alerts

Detect failures, DLQ growth, and capacity issues early.

7. OUTCOMES



Loose Coupling
Services evolve independently.



High Scalability
Handle more load and traffic efficiently.



Better Resilience
Systems withstand failures and recover gracefully.



Business Agility
Deliver value faster with reliable integrations.

6. KEY TAKEAWAYS



Choose the Right Pattern
Select sync, async, or hybrid based on business needs and tradeoffs.



Design for Independence
Use events and messages to minimize coupling between services.



Plan for Failures
Implement retries, idempotency, DLQs, and circuit breakers.



Ensure Data Consistency
Use eventual consistency, outbox pattern, and sagas when needed.



Observe Everything
Monitor queues, lag, failures, and message rates.



Secure by Design
Authenticate services and protect messages in transit and at rest.

DATA AND TRANSACTION PATTERNS RECAP · RESILIENCE AND PERFORMANCE RECAP · OBSERVABILITY RECAP



WHAT WE ESTABLISHED

- Database-per-service as the non-negotiable ownership rule
- Sagas and compensating transactions in place of two-phase commit



WHAT WE ESTABLISHED

- Resilience4j patterns for graceful degradation under dependency failure
- Caching and SLO-driven performance budgets as a discipline, not an afterthought



WHAT WE ESTABLISHED

- The three pillars of observability, correlated by trace and span IDs
- SLO-based alerting that signals user impact, not just technical thresholds

KEY TAKEAWAY Database-per-service ownership, sagas with compensating actions, and the outbox pattern are how this course achieved consistency across services without ever needing a distributed transaction. • Resilience4j's timeouts, retries, circuit breakers, and bulkheads, paired with a deliberate caching strategy, are the patterns this course relied on to keep services up when their dependencies fail. • Structured logs, Micrometer metrics, and distributed tracing, all correlated by a shared trace ID, are what this course relied on to turn an incident from a guessing game into a fast, evidence-based investigation.

DEVOPS AND CI-CD RECAP · CONTAINERIZATION AND OPENSIFT RECAP · SECURITY AND COMPLIANCE RECAP



WHAT WE ESTABLISHED

- GitOps-style pipelines with build, quality, test, security, and deployment gates
- Environment promotion instead of rebuilding artifacts per environment



WHAT WE ESTABLISHED

- Multi-stage, minimal, non-root container images as the standard build pattern
- OpenShift Routes, probes, ConfigMaps/Secrets, and HPA as the deployment platform



WHAT WE ESTABLISHED

- OAuth2/OIDC, JWT, and mTLS as the security baseline for every service
- Secrets management and audit logging as compliance requirements, not extras

KEY TAKEAWAY An automated CI/CD pipeline with quality, test, and security gates, promoting the same artifact through Dev, QA, and Production, is what this course relied on to make every change a deployable, evidenced release. • A consistent containerization pattern, deployed to OpenShift with routes, probes, ConfigMaps, and autoscaling, is what let every service in this course run identically across dev, staging, and production. • OAuth2/OIDC for users, mTLS between services, and disciplined secrets management are the controls this course relied on to satisfy the audit question every regulated system eventually faces.

GITHUB COPILOT RECAP · AGENTIC AI RECAP · MODEL CONTEXT PROTOCOL RECAP



WHAT WE ESTABLISHED

- Prompt engineering -- role, task, context, constraints, output, quality -- as the skill that matters most
- Human review remains mandatory for every AI-generated line before it ships



WHAT WE ESTABLISHED

- **The agent loop:** plan, select tools, execute, reason, respond, learn from feedback
- Guardrails -- least privilege, human approval, audit logging -- as non-negotiable design elements



WHAT WE ESTABLISHED

- Tools, resources, and prompts as the three building blocks of an MCP server
- Security and policy enforcement at the MCP boundary, not inside every agent

KEY TAKEAWAY Across this course, Copilot proved fastest as a drafting partner for boilerplate, tests, and documentation -- never as a substitute for the developer's own review, judgment, and final sign-off. • The agentic AI pattern -- understand intent, select tools, execute actions, reason over results, respond -- with human approval gating anything risky, is what this course built toward from the very first microservice. • MCP's tools, resources, and prompts, enforced behind an authentication and policy boundary, are what let this course's AI agents discover and invoke microservice capabilities safely, without hardcoding integration logic.

PRODUCTION-READINESS CHECKLIST · COMMON IMPLEMENTATION PITFALLS · RECOMMENDED MICROSERVICES ENGINEERING PRACTICES



THE CHECKLIST

- All automated tests and security scans pass; performance targets are met
- Observability and rollback are proven; runbooks and ownership are documented



PITFALLS TO AVOID

- Sharing a database across services; synchronous call chains with no timeouts
- Trusting AI-generated code without the same review bar as human code



THE PRACTICES TO KEEP

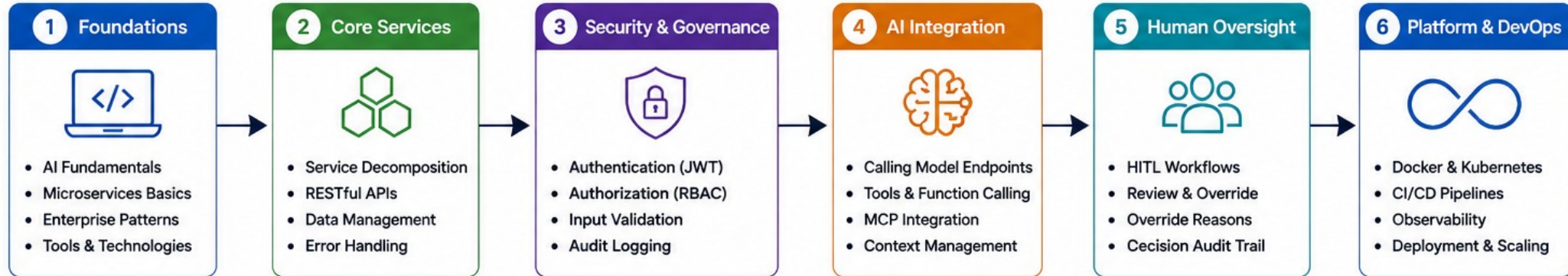
- Own your data; test your contracts; wrap every dependency in resilience policies
- Instrument everything before you need it, not after an incident

KEY TAKEAWAY A production-readiness checklist -- tests, security, performance, observability, rollback, and runbooks all signed off -- is the objective bar this course used in place of a subjective "it feels done." • The pitfalls that repeatedly undo microservices projects -- shared databases between services, missing timeouts and retries, and skipping review on AI-generated code -- are avoidable with the discipline this course taught. • Database-per-service ownership, contract testing, resilience policies on every call, and full observability are the four practices this course recommends carrying forward into any future microservices project.

Course Summary



Outcome: You can design, build, and deploy secure, enterprise-grade AI-powered microservices with authentication, audit logging, and human oversight.



Cross-Cutting Concerns



Security

Secure by Design,
Least Privilege



Compliance

Policies, Regulations,
Data Protection



Observability

Logging, Metrics,
Tracing, Alerts



Reliability

Resilience, Timeouts,
Retries, Circuit Breakers



Performance

Caching, Rate Limiting,
Load Balancing



Data Integrity

Validation, Consistency,
Idempotency

End-to-End Capstone Flow



User / Client
Request



API Gateway
& Security



Microservices
Processing



AI Model /
Tools



Human Review
(HITL)



Data Store
& Events



Audit Log
& Metrics



Response
to Client

Key Skills You Now Have



Design enterprise
microservices
architectures



Build secure
REST APIs



Integrate AI
models safely



Implement
HITL workflows



Ensure auditability,
compliance &
governance



Deploy with
CI/CD on
Kubernetes



Monitor, operate
and improve
systems



You're Ready To:



Build AI-powered
enterprise solutions



Deliver secure,
compliant systems



Collaborate in
high-performing teams



Drive innovation
with responsibility

TECHNOLOGY SELECTION DECISION GUIDE · FINAL KNOWLEDGE CHECK



HOW TO DECIDE

- This course uses Kafka for TransactionSubmitted (ordering and replay). RabbitMQ is a lecture comparison, not a lab install
- Strong consistency favors SQL; flexible schema and scale favor NoSQL



WHAT THE CHECK COVERS

- Every module topic from Day 1 fundamentals through Day 5's capstone and AI integration

KEY TAKEAWAY Choose the tool from requirements: this course's event lab is Kafka. RabbitMQ, Redis, and extra stores stay lecture. • The final knowledge check spans every module of this five-day course -- design, Spring Boot, integration, data, resilience, observability, DevOps, security, and AI -- confirming the whole arc from monolith to AI-ready microservices platform landed.

Final Knowledge Check

1 AI & Generative AI Fundamentals

- What are the key differences between traditional AI and Generative AI?
- What are large language models (LLMs)?
- What are tokens, context windows, and prompts?

2 Microservices Architecture

- What are the benefits of microservices?
- How do services communicate?
- What is the role of an API Gateway?

3 Security & Governance

- How does JWT-based authentication work?
- What is RBAC?
- Why are input validation and least privilege important?

4 Calling Models & AI Integration

- How do you call a model endpoint securely?
- What is the purpose of context management?
- How does MCP enhance integrations?

5 Human-in-the-Loop (HITL)

- When should HITL be used?
- What are override reasons?
- How is the audit trail maintained?

6 Tools, MCP & Function Calling

- What is a tool in the context of AI systems?
- How does function calling work?
- How do MCP clients, servers, and tools interact?

7 Observability & Audit Logging

- What key events should be logged?
- Which metrics indicate system health?
- How do dashboards and alerts help operations?

8 Deployment & DevOps

- What are the stages of the CI/CD pipeline?
- How do Docker and Kubernetes support deployments?
- How do you ensure zero-downtime deployments?

9 Performance, Reliability & Scalability

- How do caching and rate limiting improve performance?
- What strategies improve resilience?
- How do you plan for horizontal scaling?

10 Capstone Architecture & End-to-End Flow

- What are the main components of the capstone architecture?
- Walk through the end-to-end request flow.
- How does the system ensure security, auditability, and human oversight?

Capstone Scenarios		Success Criteria			
User Registration & Profile Management	Payment Processing Integration	Functionality All scenarios work as expected	Security Authentication, RBAC, validation in place	Reliability Resilient, fault-tolerant, and available	Performance Meets response time and load requirements
Secure Authentication & Authorization (RBAC)	Risk Assessment Service (AI-Powered)	Auditability Complete logs and audit trails	User Experience Clear, intuitive, and responsive	DevOps Excellence Automated, repeatable, and observable	Capstone Readiness Demo-ready with documentation
Customer Management (CRUD)	HITL Review & Override				
Policy Management & Search	Audit Logging & Observability				
Claim Submission & Processing	End-to-End Integration & Demo				



You're Ready!

Apply your knowledge. Build with confidence.
Deliver enterprise-grade AI-powered solutions.



Next Step: Complete the capstone demo and showcase your solution!

INDIVIDUAL AND TEAM ACTION PLANS

Turn learning into results. Define your next steps and commit to building better microservices solutions in your organization.



Learn
(Course)



Plan
(Action)



Execute
(Apply)



Improve
(Measure)



INDIVIDUAL ACTION PLAN



1. GOALS

What do I want to achieve as a result of this course?

- _____
- _____
- _____



2. APPLY & PRACTICE

What concepts or skills will I apply first?

- _____
- _____
- _____



3. PROJECTS & INITIATIVES

What projects or improvements will I contribute to?

- _____
- _____
- _____



4. LEARNING & GROWTH

What will I continue learning or explore next?

- _____
- _____
- _____



5. COMMITMENT

When will I take action and how will I track progress?

- _____
- _____
- _____



My Commitment

I will apply what I learned to deliver value, collaborate effectively, and continuously improve.

Target Date: _____



TEAM ACTION PLAN



1. TEAM GOALS

What outcomes do we want to achieve together?

- _____
- _____
- _____



2. KEY INITIATIVES

What initiatives will we prioritize?

- _____
- _____
- _____



3. ROLES & RESPONSIBILITIES

Who will do what?

- _____
- _____
- _____



4. SUCCESS MEASURES

How will we measure success?

- _____
- _____
- _____



5. TIMELINE & MILESTONES

Key milestones and target dates

- _____
- _____
- _____



Team Commitment

We will collaborate, communicate, and deliver resilient, secure, and scalable microservices solutions.

Target Date: _____

PLAN FOR SUCCESS



Start small and iterate incrementally.



Align with business priorities.



Leverage your team and community.



Use automation and good practices.



Measure, learn, and adapt.



Share knowledge and mentor others.



FOCUS AREAS FROM THE COURSE



Service Design

Design for business value, loose coupling, and independent deployment.



Security

Secure by design with strong identity, secrets management, and least privilege.



Resilience

Use resilience patterns to handle failure and maintain availability.



Observability

Implement logs, metrics, and traces for insights and faster troubleshooting.



DevOps

Automate CI/CD, testing, and deployments for speed and quality.



AI & MCP

Use AI tools and MCP to accelerate innovation responsibly.



Your knowledge is powerful.
Your actions create impact.



Commit today. Act consistently.
Build microservices that make a difference.

ADDITIONAL LEARNING RESOURCES · COURSE SUMMARY · QUESTIONS AND CLOSING DISCUSSION



WHERE TO GO NEXT

- Official framework and platform documentation for Spring Boot and OpenShift
- The capstone repository itself, as a working reference for every pattern covered



THE ARC OF THE COURSE

- Design and build services -> integrate and secure them -> operate and observe them -> extend them with AI
- The capstone proved every pattern together, on one real platform



CLOSING THE COURSE

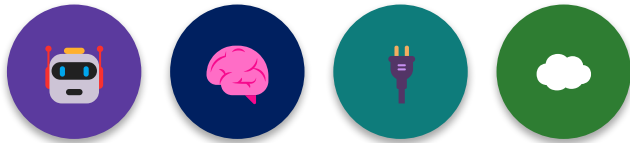
- Open floor for questions on any module, the capstone, or how to apply this at work
- Thank you for completing MD287 -- go build resilient, secure, AI-ready microservices

KEY TAKEAWAY Official Spring, OpenShift/Kubernetes, and OWASP documentation, alongside the capstone codebase built during this course, are the resources to return to when a real project raises a question this course only had time to introduce. • This course took the platform from a single Spring Boot service to a resilient, secure, observable, containerized, AI-ready microservices platform running on OpenShift -- the full arc this capstone demonstrated end-to-end. • An open discussion for remaining questions, edge cases from your own projects, and any part of the microservices, DevOps, or AI-agent material worth revisiting before applying it at work.

LAB 5

Lab 5 — Risk Assessment Service

Risk Assessment Service + OpenShift AI integration; capstone demonstration.



Lab 5 Overview - Risk Assessment Service

1. Lab Objective

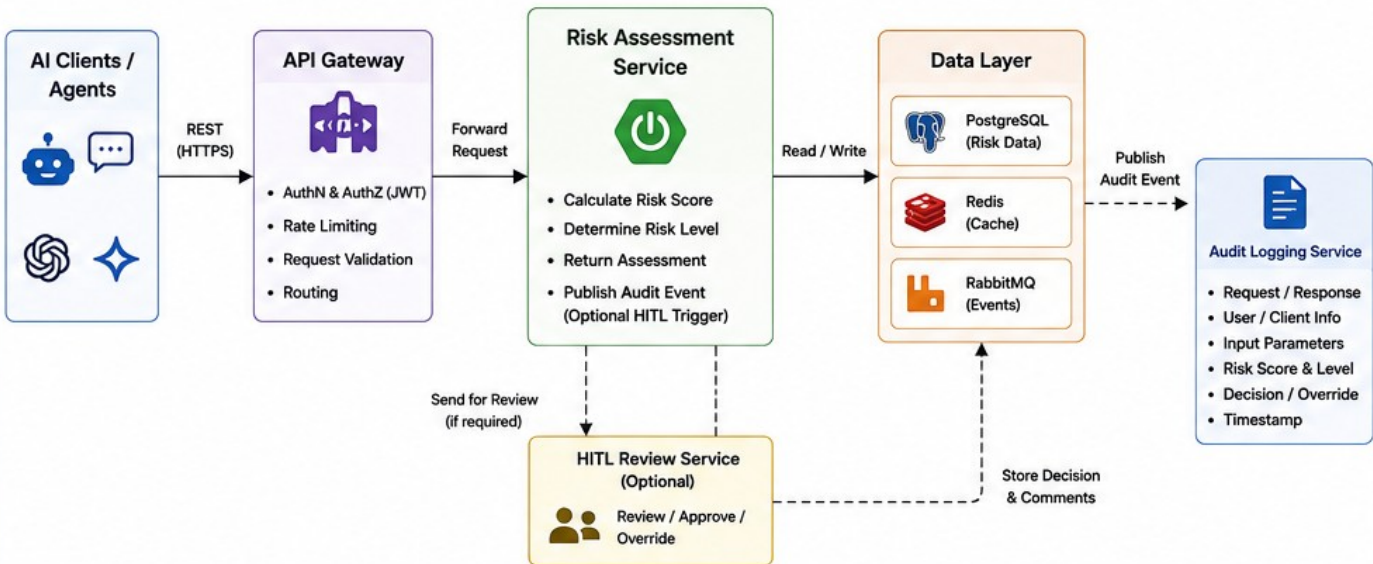


Build and expose a Risk Assessment microservice as an AI-callable tool with authentication, HITL (optional), and audit logging.

3. Endpoints (Examples)

Method	Endpoint	Description
POST	/api/v1/risk/assess	Assess risk for a customer
GET	/api/v1/risk/{id}	Get assessment by ID
GET	/api/v1/risk/history	Get assessment history
GET	/api/v1/risk/health	Health check

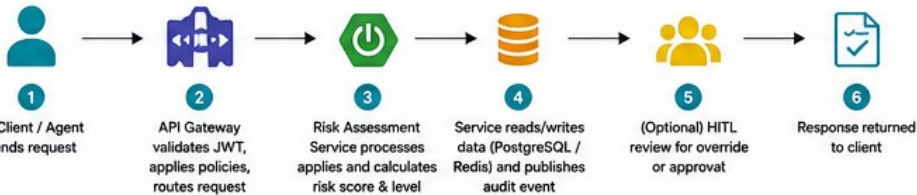
2. Solution Architecture



4. Tech Stack

- Spring Boot
- PostgreSQL
- Redis
- RabbitMQ
- JWT (Auth)
- Docker
- Kubernetes

5. Request Flow



6. Lab Deliverables

- ✓ Risk Assessment microservice (Spring Boot)
- ✓ Endpoints to assess and retrieve risk
- ✓ Integration with PostgreSQL & Redis
- ✓ JWT-secured API via API Gateway
- ✓ Audit logging of all requests & decisions
- ✓ (Optional) HITL review & override workflow
- ✓ Dockerized service deployed to Kubernetes
- ✓ Test collection (Postman) and documentation

7. Success Criteria

- ✓ Assess risk and return correct score & level
- ✓ Secure access with valid JWT
- ✓ All requests and responses are audited
- ✓ (Optional) HITL workflow functions correctly
- ✓ Service is containerized and deployed
- ✓ All tests pass (functional + security)

8. Sample Risk Assessment Request / Response

Request

```
{  "customerId": "CUST12345",  "attributes": {    "age": 45,    "income": 85000,    "creditScore": 720,    "existingExposure": 15000  }}
```

Response

```
{  "assessmentId": "a3d9f6c2-9b42-4d1b-8c1e-2f3e5c8a1b7d",  "attributes": {    "CUST12345",    "riskScore": 78,    "riskLevel": "HIGH",    "factors": ["creditScore", "existingExposure"],    "recommendation": "Manual review recommended",    "timestamp": "2025-05-09T14:30:22Z"  }}
```

9. Key Concepts Practiced

- Secure APIs
- RBAC & JWT Auth
- Audit Logging
- HITL Workflow
- Microservices Design
- Containerization & Deployment

LAB 5 — RISK ASSESSMENT SERVICE — OVERVIEW

Day 5 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Risk Assessment Service on port 8083
- Its own database (risk_db on host port 5435)
- A consumer on Kafka topic transactions.submitted (group risk-assessment-service)
- Model call to the pre-deployed OpenShift AI endpoint (same contract as the local stand-in on :8090)
- Risk Assessment deployed to the assigned OpenShift project (Route evidence)
- Policy outcomes APPROVE, HOLD, DECLINE
- Model timeout or HTTP 500 → HOLD MODEL_UNAVAILABLE (never auto-approve)
- Human review on HOLD only (risk.write)
- **Audit fields:** model name/version, score, policy version, correlation id

KEY TAKEAWAY Lab 5 builds Risk Assessment Service -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 5 — RISK ASSESSMENT SERVICE — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 5)

Day 5 idea	How it appears in this lab
Model endpoint	ModelClient POST with Bearer API key. Local stand-in :8090 for coding; required call to the pre-deployed OpenShift AI Route (MD287_MODEL_ROUTE).
Safe fallback	Timeout or 5xx → HOLD, not APPROVE
Deterministic policy	Java PolicyEngine owns APPROVE / HOLD / DECLINE
Human review	HOLD rows stay PENDING until a reviewer posts APPROVE or DECLINE
Audit	Persist modelName, modelVersion, modelScore, policyVersion, correlationId
MCP	Exercise 5.3 worksheet only — auth, HITL, audit if a tool wrapped this API

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 5 — RISK ASSESSMENT SERVICE — LAB STEPS

8 steps, worked in order.

LAB STEPS

- Step 1 — Start local backing services and prove the pre-deployed model
- Step 2 — Confirm the starter still refuses to assess
- Step 3 — Implement PolicyEngine (Exercise 5.2)
- Step 4 — Safe model fallback

LAB STEPS (CONTINUED)

- Step 5 — Protect the API (scopes)
- Step 6 — Run the service and publish sample events
- Step 7 — Integrate with Account + Transaction events
- Step 8 — Deploy Risk Assessment to OpenShift and MCP (Exercise 5.3)

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 5 — RISK ASSESSMENT SERVICE — SUCCESS CRITERIA

You are done when every box below is ticked.



SUCCESS CRITERIA

- mvn test passes
- Sample events produce APPROVE / HOLD / DECLINE / MODEL_UNAVAILABLE / HIGH_VALUE
- Duplicate event does not create a second row
- Reviewer can complete HOLD; ops cannot



SUCCESS CRITERIA (CONTINUED)

- Logs stay synthetic and secret-free
- Pre-deployed OpenShift AI Route /v1/health is UP (MD287_MODEL_ROUTE)
- Risk Assessment deployed; Route readiness 200
- MCP worksheet filled

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.